Técnicas de Inteligência Artificial

PL

Licenciatura em Engenharia e Gestão de Sistemas de Informação

Filipe Portela

Exercícios

Exercício 1 - Resolução

```
% a)
- supervisiona(rita,joao,c).
- supervisiona(rita,rui,c).
- supervisiona(joao,iva,c).
- supervisiona(vitor,rui,s).
- supervisiona(vitor,tomas,s).
- supervisiona(tomas,ivo,s).
- supervisiona(rui,miguel,s).

- % b)
- chefe(rita,c).
- chefe(vitor,s).

% c)
- mesmonivel(X,Y):-
  supervisiona(X,Z,_),supervisiona(Y,Z,_), X\==Y.

% d)
- trabalha(X,E):- chefe(X,E);supervisiona(X,_,E).
% e)
- temchefe(X,Y):-supervisiona(X,_,E),chefe(Y,E).

% f)
- acima(X,Y,E):- supervisiona(X,Y,E).
- acima(X,Y,E):- supervisiona(X,Z,E),acima(Z,Y,E).
```

```
% g)
- sexo(joao,m).
- sexo(rui,m).
- sexo(iva,f).
- sexo(rita,f).
- sexo(tomas,m).
- sexo(ivo,m).
- sexo(miguel,m).
- sexo(vitor,m).
- sexodif(X,Y):- sexo(X,S1),sexo(Y,S2),S1\==S2.
- perigoassedio(X,Y):-
  sexodif(X,Y),(supervisiona(X,Y,_);supervisiona(Y
  ,X,_)).

%questões
- q1:- supervisiona(rita,joao,_).
- q2(X):- supervisiona(rita,X,_).
- q3a(X):- mesmonivel(joao,X).
- q3b(X):- mesmonivel(rui,X).
- q4(X):- acima(ivo,X,_).
- q5(X):- temchefe(X,rita).
- q6(X):- trabalha(X,E),trabalha(X,E1),E\==E1.
- q7(X):- perigoassedio(joao,X).
- q8(X):- trabalha(X,E), \+
  (trabalha(X,E1),E\==E1).
```

Exercícios

Exercício 1 - Resolução

%a

```
trabalhapara(joao,  rita,  c).
trabalhapara(rui,   rita,  c).
trabalhapara(iva,   joao,  c).
trabalhapara(miguel, rui,  s).
trabalhapara(rui,   vitor, s).
trabalhapara(tomas, vitor, s).
trabalhapara(ivo,   tomas, s).
```

%b

```
chefe(rita,  c).
chefe(vitor, s).
```

%c

```
mesmonivel(X, Y) :-
    X \== Y,
    trabalhapara(X, Chefe, _),
    trabalhapara(Y, Chefe, _).
```

%d

```
empregado(X, E) :-
    chefe(X, E).
empregado(X, E) :-
    trabalhapara(X, _, E).
empregado(X, E) :-
    trabalhapara(_, X, E).
```

%e

```
temchefe(X, Y) :-
    empregado(X, E),
    chefe(Y, E).
```

%f

```
acima(X, Y, E) :-
    trabalhapara(X, Y, E).
acima(X, Y, E) :-
    trabalhapara(X, Z, E),
    acima(Z, Y, E).
```

%g

```
sexo(joao,   m).
sexo(rui,    m).
sexo(iva,    f).
sexo(rita,   f).
sexo(tomas,  m).
sexo(ivo,    m).
sexo(miguel, m).
sexo(vitor,  m).
sexodif(X, Y) :-
    sexo(X, S1),
    sexo(Y, S2),
    S1 \== S2.
perigoassedio(X, Y) :-
    sexodif(X, Y),
    ( acima(X, Y, _) ; acima(Y, X, _) ).
```

%h

```
q1 :- acima(joao, rita, _).
q2(X) :- acima(X, rita, _).
q3a(X) :- mesmonivel(joao, X).
q3b(X) :- mesmonivel(rui, X).
q4(X) :- acima(ivo, X, _).
q5(X) :-
    chefe(rita, E),
    empregado(X, E).
q6(X) :-
    empregado(X, E1),
    empregado(X, E2),
    E1 \== E2.
q7(X) :- perigoassedio(joao, X).
q8(X) :-
    empregado(X, E),
    \+ (empregado(X, E2), E2 \== E).
```

Exercícios

Exercício 1 - Resolução

Abordagem Alternativa

```
supervisiona(rita,joao).
supervisiona(rita,rui).
supervisiona(joao,iva).
supervisiona(vitor,rui).
supervisiona(vitor,tomas).
supervisiona(tomas,ivo).
supervisiona(rui,miguel).
chefe(rita,c).
chefe(vitor,s).
trabalha(rita, c).
trabalha(vitor, s).
trabalhapara(X, Y, Z) :-
    supervisiona(Y, X),
    (   trabalha(Y, Z)
    ;   trabalhapara(Y, _, Z)
    ).
chefe(E, Z) :-
    trabalhapara(E, _, Z),
    \+ trabalhapara(_, E, Z).
```

SWISH File Edit Examples Help

125 users online

Search

Program

```
%a)
trabalhapara(joao,  rita,  c).
trabalhapara(rui,   rita,  c).
trabalhapara(iva,   joao,  c).
trabalhapara(miguel, rui,  s).
trabalhapara(rui,   vitor, s).
trabalhapara(tomas, vitor, s).
trabalhapara(ivo,   tomas, s).
% (b)
chefe(rita,  c).
chefe(vitor, s).
% (c)
mesmonivel(X, Y) :-
    X \== Y,
    trabalhapara(X, Chefe, _),
    trabalhapara(Y, Chefe, _).
% (d)
empregado(X, E) :-
    chefe(X, E).
empregado(X, E) :-
    trabalhapara(X, _, E).
empregado(X, E) :-
    trabalhapara(_, X, E).
% (e)
temchefe(X, Y) :-
    empregado(X, E),
    chefe(Y, E).
% (f)
acima(X, Y, E) :-
    trabalhapara(X, Y, E).
acima(X, Y, E) :-
    trabalhapara(X, Z, E),
    acima(Z, Y, E).
% (g)
sexo(joao,   m).
sexo(rui,    m).
sexo(iva,    f).
sexo(rita,   f).
sexo(tomas,  m).
sexo(ivo,    m).
sexo(miguel, m).
sexo(vitor,  m).
sexodif(X, Y) :-
    sexo(X, S1),
    sexo(Y, S2),
    S1 \== S2.
perigoassedio(X, Y) :-
    sexodif(X, Y),
```

q1; q2(X); q3a(X); q3b(X); q4(X); q5(X); q6(X); q7(X); q8(X).

true 1

X = joao
X = rui
X = iva
X = joao
X = rui
X = joao
X = rui
X = rui
X = tomas
X = tomas
X = vitor
X = rita
X = joao
X = rui
X = iva
X = rita
X = rita
X = joao
X = rui
X = rui
X = rui
X = rui
X = iva
X = rita
X = rita
X = vitor
X = joao
X = iva
X = miguel

```
?- q1;
   q2(X);
   q3a(X);
   q3b(X);
   q4(X);
   q5(X);
   q6(X);
   q7(X);
   q8(X).
```

Examples History Solutions

table results Run!

Listas

- Em Prolog, uma lista é uma estrutura de dados que contém uma sequência ordenada de elementos, separados por vírgulas e delimitados por parênteses retos [].

- As listas em Prolog são homogêneas, ou seja, todos os elementos devem ter o mesmo tipo.
  - Por exemplo, a lista [1, 2, 3, 4, 5] contém cinco elementos inteiros. Já a lista [a, b, c, d, e] contém cinco elementos átomos.

- Os elementos de uma lista são acedidos separando-se a lista em duas partes: a **cabeça** (*head*) e a **cauda** (*tail*).
  - A cabeça é o primeiro elemento da lista e a cauda é o restante da lista, sem o primeiro elemento.
  - Sintaxe : [H | T]

```
% empty list []
% singleton list [a]
% 2 element list [hello, world]
% 3 element list [[1,2,3,4], p, this]
% nested list (3 level nesting)  [[[1, 2], [3, 4]], [[a, b], [x, y]]].
% L is a list and a is a single element. [a | L]
```

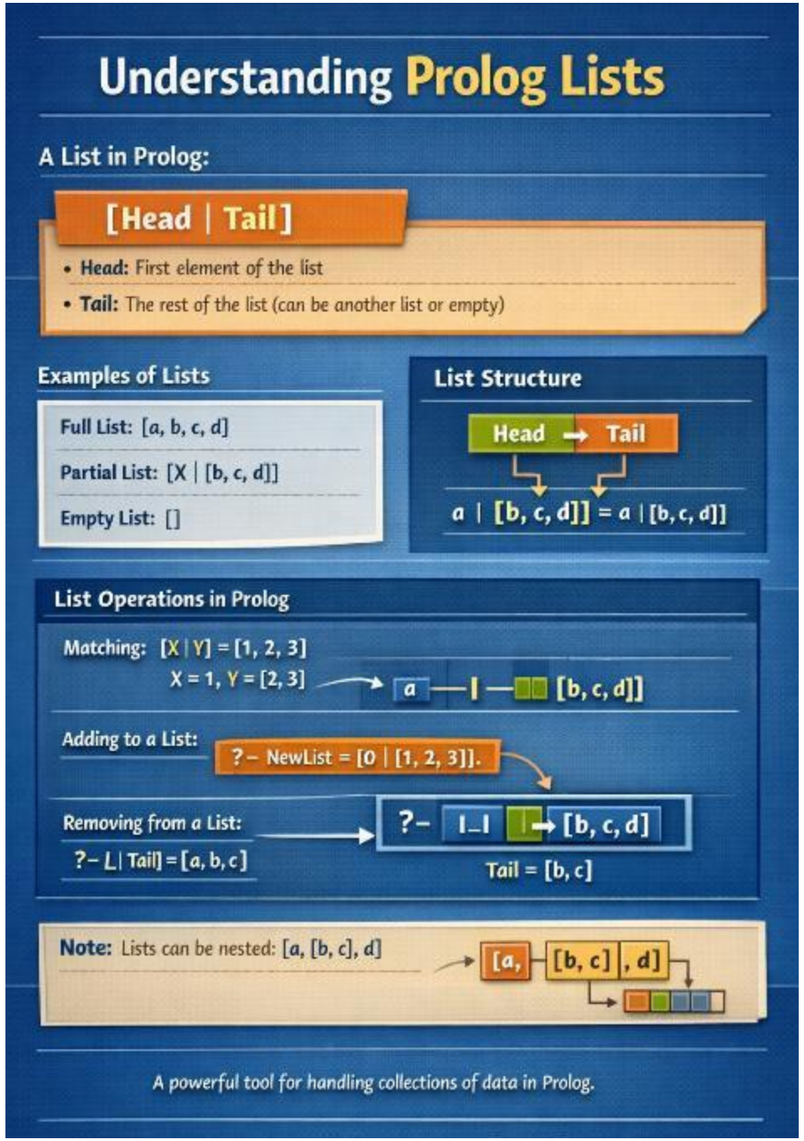
```
?- [Head | Tail] = [1, 2, 3, 4, 5].
Head = 1,
Tail = [2, 3, 4, 5].
```

- Operações Listas | Links Prolog

Listas

- As principais operações que podem ser realizadas com listas em *Prolog* são:
  - **Criar uma lista**: uma lista pode ser criada diretamente, especificando os seus elementos dentro de [], como em [1, 2, 3] ou [a, b, c].
  - **Concatenar listas**: é possível unir duas ou mais listas em uma única lista, usando o operador |, como em L = [1, 2, 3] | [4, 5, 6], que resulta na lista L = [1, 2, 3, 4, 5, 6].
  - **Aceder ao elemento de uma lista**: os elementos de uma lista podem ser acedidos pelo seu índice, usando o operador nth0 ou nth1. Por exemplo, nth0(2, [a, b, c, d, e], X) retorna X = c.
  - **Adicionar ou remover elementos**: novos elementos podem ser adicionados ao início ou fim de uma lista usando os operadores | ou append, enquanto o operador *delete* é usado para remover elementos de uma lista
  - **Verificar conteúdo**: é possível verificar se um elemento pertence a uma lista usando o operador *member*, como em member(a, [a, b, c, d, e]).
  - **Verificar tamanho de uma lista**: o número de elementos em uma lista pode ser obtido usando o operador *length*, como em length([a, b, c, d, e], N), que retorna N = 5
  - **Subdividir uma lista**: uma lista pode ser dividida em duas partes usando o operador *split*, que separa a lista em um prefixo e um sufixo em torno de um determinado elemento.

Listas



- Exemplos

```
?- [One, Two | T] = [apple, sprouts, fridge, milk].
    One = apple
    Two = sprouts
    T = [fridge, milk]

?- [a|[b|[c|[d|[]]]]] = [a,b,c,d].
yes
```

```
first([X | _ ], X).
second([ _, X | _ ], X).
third([ _, _, X | _ ], X).
```

Listas

- **:-dynamic**
  - Utilizado para declarar que um predicado pode ser modificado durante a execução, ou seja, que seus factos podem ser adicionados, removidos ou modificados dinamicamente durante a execução do programa.
  - Sempre que uma lista tiver de ser alterada, a mesma tem de ser declarada como dinâmica
    - `:-dynamic(lista1/1) %lista 1 é dinâmica e recebe um parâmetro`
    - `:-dynamic(lista1/2) %lista 2 é dinâmica e recebe dois parâmetro`

- **Funções Recursivas**
  - Técnica utilizada para definir predicados que se chamam a si próprios, permitindo que sejam processados dados complexos de forma eficiente garantindo que todas as possibilidades são abordadas

```
tamanho([], 0).
tamanho([_|Tail], N) :-
    tamanho(Tail, N1),
    N is N1 + 1.

?- tamanho([1, 2, 3, 4, 5], N).
N = 5.
```

Exercícios

Exercício 2

Represente em **Prolog** os seguintes predicados genéricos sobre listas

1. **adiciona(X,L1,L2)** – onde L2 é a lista que contém o elemento X e a lista L1.
   - Testar este predicado no interpretador Prolog, executando:
     - ?- adiciona(1,[2,3],L).
     - ?- adiciona(X,[2,3],[1,2,3]).
2. **membro(X,L)** – que é verdadeiro se X pertencer à lista L. Testar com:
   - ?- membro(b,[a,b,c]).
   - ?- membro(X,[a,b,c]). % carregar em ;
   - ?- findall(X,membro(X,[a,b,c]),L).
3. **apaga(X,L1,L2)** – onde L2 é a lista L1 sem o elemento X. Testar com:
   - ?- apaga(a,[a,b,a,c],L).
   - ?- apaga(a,L,[b,c]).
4. **concatena(L1,L2,L3)** – onde L3 é resultado da junção das listas L2 e L1. Testar:
   - ?- concatena([1,2],[3,4],L).
   - ?- concatena([1,2],L,[1,2,3,4]).
   - ?- concatena(L,[3,4],[1,2,3,4]).
5. **sublista(S,L)** – que é verdadeiro se S for uma sublista (estiver contido em) de L.
   - Testar:
     - ?- sublista([rui,ana],[rita,rui,ana,ivo]).
     - ?- sublista(L,[rita,rui,ana,ivo]). % carregar em ;
6. **comprimento(X,L)** – onde X é o número de elementos da lista L. Testar com:
   - ?- comprimento(X,[a,b,c]).
7. **max(X,L)** – onde X é o valor máximo da lista L (assumir que L contém somente
   - números). Testar com:
   - ?- max(X,[3,2,1,7,4]).
8. **media(X,L)** – onde X é o valor médio da lista L (assumir que L contém somente números). Testar com:
   - ?- media(X,[1,2,3,4,5]).
9. **nelem(N,L,X)** – onde N é um número e X é o elemento da lista L na posição L.
   - Por exemplo (testar com):
     - ?- nelem(2,[1,2,3],2).
     - ?- nelem(3,[1,2,3],X).
     - ?- nelem(4,[a,b,c,d,e,f,g],X).

Nota: o swi-prolog já tem diversos predicados (no module lists) para manipulação de listas. Neste exercício, não é para utiliza-los, mas para futuro uso aqui se listam as equivalências destas alíneas: 2) delete 3) member 4) append 7) max_list e 9) nth0 ou nth1. <u>Estes predicados tal como o findall apenas serão trabalhados na próxima aula.</u>

Exercícios

Exercício 2

Represente em **Prolog** os seguintes predicados genéricos sobre listas

1. **adiciona(X,L1,L2)** – onde L2 é a lista que contém o elemento X e a lista L1.
   - Testar este predicado no interpretador Prolog, executando:
     - ?- adiciona(1,[2,3],L).
     - ?- adiciona(X,[2,3],[1,2,3]).
2. **membro(X,L)** – que é verdadeiro se X pertencer à lista L. Testar com:
   - ?- membro(b,[a,b,c]).
   - ?- membro(X,[a,b,c]). % carregar em ;
   - ?- findall(X,membro(X,[a,b,c]),L).
3. **apaga(X,L1,L2)** – onde L2 é a lista L1 sem o elemento X. Testar com:
   - ?- apaga(a,[a,b,a,c],L).
   - ?- apaga(a,L,[b,c]).

```
% Como começar? Será dinâmico?
:-dynamic(lista1/1).

% 2.1 adiciona(X,L1,L2) – onde L2 é a lista que contém o elemento X e a lista L1.
adiciona(X,L,[X|L]). %quais os resultados? Adiciona(1,[2,3],L) e ?-
,[1,
```

adiciona(1,[2,3],L).

L = [1, 2, 3]

adiciona(X,[2,3],[1,2,3]).

X = 1

```
% 2.2 membro(X,L) – que é verdadeiro se X pertencer à lista L.
membro( X, [X|_] ).
membro( X, [_|R] ) :- membro( X, R ). %quais os resultados?
```

membro(b,[a,b,c]).

true

membro(X,[a,b,c]).

X = a
X = b
X = c

findall(X,*membro*(X,[a,b,c]),L).

L = [a, b, c]

```
:-dynamic(lista1/1).
adiciona(X,L,[X|L]).

membro( X, [X|_] ).
membro( X, [_|R] ) :- membro( X, R ).
```

```
% 2.3. apaga (sem repetidos)
apaga(X,[X|R],R).
apaga(X,[Y|R1],[Y|R2]):- apaga(X,R1,R2).
```

apaga(c,[a,b,a,c,d], L).

L = [a, b, a, d]

Exercícios

Exercício 2

Represente em **Prolog** os seguintes predicados genéricos sobre listas

1. **adiciona(X,L1,L2)** – onde L2 é a lista que contém o elemento X e a lista L1.
   - Testar este predicado no interpretador Prolog, executando:
     - ?- adiciona(1,[2,3],L).
     - ?- adiciona(X,[2,3],[1,2,3]).
2. **membro(X,L)** – que é verdadeiro se X pertencer à lista L. Testar com:
   - ?- membro(b,[a,b,c]).
   - ?- membro(X,[a,b,c]). % carregar em ;
   - ?- findall(X,membro(X,[a,b,c]),L).
3. **apaga(X,L1,L2)** – onde L2 é a lista L1 sem o elemento X. Testar com:
   - ?- apaga(a,[a,b,a,c],L).
   - ?- apaga(a,L,[b,c]).
4. **concatena(L1,L2,L3)** – onde L3 é resultado da junção das listas L2 e L1. Testar:
   - ?- concatena([1,2],[3,4],L).
   - ?- concatena([1,2],L,[1,2,3,4]).
   - ?- concatena(L,[3,4],[1,2,3,4]).
5. **sublista(S,L)** – que é verdadeiro se S for uma sublista (estiver contido em) de L.
   - Testar:
     - ?- sublista([rui,ana],[rita,rui,ana,ivo]).
     - ?- sublista(L,[rita,rui,ana,ivo]). % carregar em ;
6. **comprimento(X,L)** – onde X é o número de elementos da lista L. Testar com:
   - ?- comprimento(X,[a,b,c]).
7. **max(X,L)** – onde X é o valor máximo da lista L (assumir que L contém somente
   - números). Testar com:
   - ?- max(X,[3,2,1,7,4]).
8. **media(X,L)** – onde X é o valor médio da lista L (assumir que L contém somente números). Testar com:
   - ?- media(X,[1,2,3,4,5]).
9. **nelem(N,L,X)** – onde N é um número e X é o elemento da lista L na posição L.
   - Por exemplo (testar com):
     - ?- nelem(2,[1,2,3],2).
     - ?- nelem(3,[1,2,3],X).
     - ?- nelem(4,[a,b,c,d,e,f,g],X).

Pede ajuda ao GEM

Exercícios

Exercício 2 – Teste dos predicados

- Exercício de teste (disponível no GEM)

- **Base de dados:**
  - empresa(c,[rita,joao,rui,iva]).
  - empresa(s,[vitor,rui,tomas,ivo,miguel]).
  - chefes([rita,vitor]).

- **Perguntas**

  a) **chefe_da_empresa(E,C) -** O chefe é o **primeiro elemento da lista** da empresa.

  b) **n_colaboradores(E,N) -** Número total de colaboradores da empresa.

  c) **pertence_empresa(X,E) -** Verificar se um colaborador pertence à empresa.

  d) **lista_sem_chefe(E,L) -** Criar uma lista com todos os colaboradores exceto o chefe.

  e) **trabalha_em_duas_empresas(X) -** Um colaborador pertence às duas listas.

  f) **total_empresas(X,N) -** Número de empresas onde X aparece.

- **Nota:** Podes fazer outras perguntas e criar o teu próprio enunciado para testar o conhecimento do exercício 2 (por exemplo, adicionar um novo colaborador)

Prolog

Documentação de suporte

- **Drive**
  - Livros, Slides Tutorial
- **Complementos**
  - Tutorial
  - Exemplos